

APE-Eureka: Reward Generation via Associative Priors Encapsulation

Anonymous submission

Abstract

Designing reward functions with large language models (LLMs) has emerged as a promising alternative to fixed skill libraries in guiding robotic behavior. However, most existing approaches primarily focus on direct task completion, overlooking how humans often rely on analogical reasoning to generalize and transfer skills. Building on this observation, we introduce **Associative Prior Encapsulation Eureka (APE Eureka)**, which uses synonymous associations to retrieve related tasks and extract useful prior knowledge. This knowledge, aligned with the original task goals and distilled from beneficial prior experiences, serves as auxiliary rewards to augment the primary reward function. To our knowledge, this is the first attempt to augment reward objectives in LLM-based reward generation using analogical priors. Experimental results across multiple MuJoCo environments show that our APE-Eureka improves Eureka’s performance by over 20%, and yields smoother, more human-like motions—all without requiring additional human feedback.

1 INTRODUCTION

The emergence of Large language models (LLMs) have broadened the ways to build instruction-following robots. Using skill primitives is an early and widely studied approach involving LLMs. In this approach, the limited adaptability of primitives quickly becomes a bottleneck (Xie et al. 2023; Yu et al. 2023). Generating reward functions using LLMs is considered a key method to mitigate this issue, and has thus attracted considerable attention. Recent studies have systematically examined various dimensions of task solving. For instance, when a task is seen as a set of sub-goals, algorithms can be designed through task decomposition and reach the goal step by step (Xie et al. 2023; Ryu et al. 2024; Ding 2024; Davidson et al. 2025; Chu et al. 2023). Alternatively, if the task is framed as adapting existing models or functions, rewards can be generated by tuning parameters or structures (Zeng, Mu, and Shao 2024; Yu et al. 2023; Hazra et al. 2025; Sha et al. 2023). These proxies serve as tools for exploration. Moreover, when reward generation is treated as an engineering optimization problem, the code can be improved using standard optimization techniques (Ma et al. 2023; Sun et al. 2024), leading to open-ended exploration.

Current LLM-based reward-generation methods rely on feedback frameworks that allow adaptation. Most decide

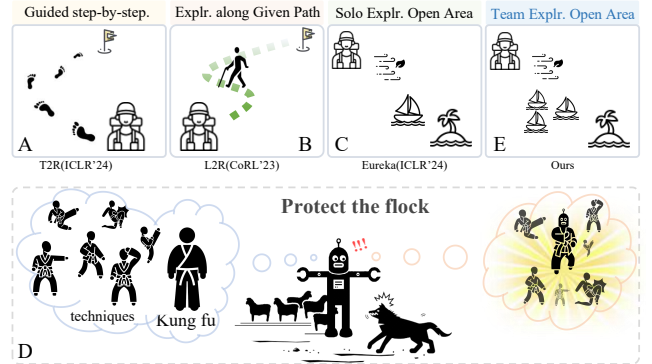


Figure 1: **Research Recap** - Fig. (A), (B), and (C) present the existing ways to generate rewards. **Motivation** - Fig. (D) shows our motivation. Certain “aha” moments yield extra guidance that helps complete the task. **Our Goal** - Fig. (E): we aim to leverage skill insights to augment task objectives to benefit reward generation.

which feedback to use and how to apply it, yet focus only on task completion. Human reasoning, by contrast, starts from familiar concepts. As shown in Fig. 1, stepping back and relating the problem to known ideas broadens the set of viable strategies. Direct problem-solving without analogy has two drawbacks. First, it limits the LLM to the reasoning and code required by the prompt, leaving outside knowledge unused and depriving the reward model of richer signals. Second, although recent work varies the reward signals, it still optimizes a single fixed objective. Without higher-level criteria, the agent explores a vast policy space without guidance and may converge to biased or incoherent strategies.

Human analogical thinking suggests a new direction: how can we harness it to generate high-quality rewards? Current priors focus on the joint distribution of actions and observations (Xu, Veloso, and Song 2022; Tang, King, and Xu 2025; Biza et al. 2021). They ignore human expertise, limiting the conceptual links the agent can form and forcing it to rely on the examples present in the dataset. Human prior knowledge remains unused in LLM reward design.

To harness analogical association, we propose the **Associative Prior Encapsulation (APE) Eureka** method. It

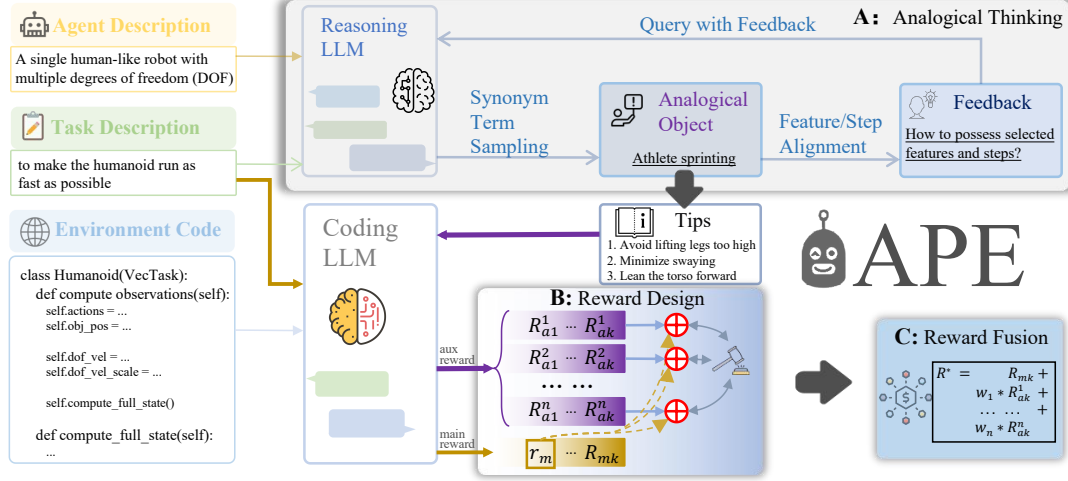


Figure 2: Overview of our framework. Module **A** retrieves skill insights for similar tasks through semantic association. These insights are helpful but lack an evaluation mechanism. Module **B** generates rewards, and module **C** fuses them, producing a well-defined reward function that improves agent performance.

augments the objective with external prior knowledge. First, an LLM performs synonym association on the key features, generating related concepts. These concepts are then grounded into executable behaviors that guide the agent. All insights follow same beneficial prior, ensuring compatibility and safe augmentation. Fig. 1 shows that when repelling enemies evokes martial arts, the robot learns concrete maneuvers that improve combat performance.

By detecting analogous tasks, the system mines extensive knowledge from LLMs. Unlike chain-of-thought reasoning, the retrieved associations are intentionally discontinuous, echoing the non-linear structure of human thought. This perspective parallels the claim in (Cao et al. 2024) that task-specific information is essential for reward generation. Likewise, LaCLIP (Fan et al. 2023) demonstrates that augmenting textual descriptions, much like image augmentation, improves CLIP performance. We believe this approach establishes a foundation for systematically augmenting reward objectives. Our contributions are primarily threefold:

- Our method augments reward objectives by drawing analogies to analogous tasks or activities.
- Associative thinking mobilizes a broader range of the LLM’s internal knowledge during code generation, offering a useful alternative to conventional task decomposition and template-based control.
- Empirical results show that our method improves Eureka’s success rate by more than 20% and yields rewards that correlate more closely with human judgments.

2 Related work

2.1 Planning

Many studies improve agent usability by engaging LLMs in predefined skill library. The SayCan project (Ahn et al. 2022) assesses command feasibility based on agent capabilities. RoboTool (Xu et al. 2023) leverages existing tools for

task completion. Replan (Skreta et al. 2024) addresses task re-planning for unexpected situations. SayPlan (Rana et al. 2023) uses preset maps to solve problems. TidyBot (Wu et al. 2023) learns user preferences for personalized service. RoCo (Mandi, Jain, and Song 2024) assigns LLMs to robots for task strategy discussions. LLMs control skillful movements and complete complex tasks in gaming (Liang et al. 2024a; Wang et al. 2023a; Li et al. 2024). These studies recognize self-uncertainty and seek external help when needed (Ren et al. 2023). Research also addresses outdated information and answer fabrication (Vemprala et al. 2023). SwiftSage (Lin et al. 2023) enables instant reactions and anomaly monitoring by mimicking human thinking patterns of fast and slow thinking. Most of these methods focus on the adaptive decision-making of embodied intelligence in different constrained scenarios.

2.2 Rewards Design

Research on generating RL reward functions using LLMs can be categorized into four distinct approaches. 1) *Focusing on the task process with LLMs*. The VoxPoser framework (Huang et al. 2023) uses LLMs to interpret environmental and command information, creating affordance and constraint maps that guide robotic arm path planning. Other studies leverage LLMs to generate sub-goals that shape reward functions (Bhambri et al. 2024; Wu 2024; Singh, Bhattacharyya, and Namboodiri 2024; Singh and Namboodiri 2023), thereby improving task success rates and convergence speeds. Among them, Text2Reward (Xie et al. 2023), CurricuLLM (Ryu et al. 2024), MobileAgent (Ding 2024), goalasreward (Davidson et al. 2025) and LgTS (Shukla et al. 2024) use LLMs to decompose tasks and generate dense rewards. LLMs also provide real-time feedback, as seen in Lafite-RL (Chu et al. 2023). Similarly, other methods utilize LLMs for human-robot dialogues and logic reasoning to guide task resolution (Kwon et al. 2023; Carta et al. 2022;

Du et al. 2023). In EurekaVerse (Liang et al. 2024b), environment difficulty is progressively increased to enhance the learning process. 2) *Designing reward templates or using parameterized models* enables LLMs to autonomously adjust parameters based on feedback (Zeng, Mu, and Shao 2024). Similarly, in Language2Rewards (Yu et al. 2023) and LanguageMPC (Sha et al. 2023), LLMs interact with MPC parameters to manage task execution via text commands and observations. REvolve (Hazra et al. 2025) 3) *Directly generate reward functions and iteratively refine them*. The Eureka algorithm (Ma et al. 2023) designs reward functions that outperform human experts by analyzing task information and environmental code, forming an open-set reward space. The CARD (Sun et al. 2024) approach also directly generates reward functions and proposes a trajectory-based feedback optimization strategy. 4) *Enhancing LLM focus on the feedback loop for valuable response pathways*. For example, in prompt2walk (Wang et al. 2023b), predefined pathways based on observations and action key points enable LLMs to generate precise robotic control commands. In SayTap (Tang et al. 2023), pathways using natural language instructions and foot contact patterns allow LLMs to produce robust gait control commands. Research (Narin 2024) also highlights the benefits of diverse feedback modalities. GROVE (Cui et al. 2025) provides reward signals using visual information. LLM-driven reward generators rely on feedback loops to decide which cues to incorporate and how to weigh them, yet they remain fixated on immediate task success and fail to exploit prior knowledge.

2.3 In-Context Methods

Expanding LLMs’ diverse common-sense knowledge as contextual assistance for task completion has also garnered widespread attention. COWP (Ding et al. 2023) highlights the value of practical concrete information, such as checking if a cup is empty before pouring tea. Step-Back Prompting (Zheng et al. 2024) incorporates domain knowledge for macrolevel thinking. In contrast, our method starts from an associative priors perspective that leverages associative results as clues to gather prior knowledge, enabling greater leaps in reasoning. Some studies focus on how prior helps in RL. Some methods transfer the demonstration to the new task. ASPIRe (Xu, Veloso, and Song 2022), CEIP (Tang, King, and Xu 2025) and Action Priors (Biza et al. 2021) use state-action pairs as a prior to guide exploration. Other methods treat human preferences as an environment prior. In preference-based RL, PRIOR (Verma and Metcalf 2022; Verma and Susa 2024) predicts useful preferences to cut manual labeling. BehaviorPF (Tirumala et al. 2020) reuses an existing policy as prior for new tasks, helping to reduce the search space. Instead, we derive priors from task insights and use them to reduce the action search space in RL.

3 Method

3.1 Framework Overview

Analogical thinking, a common human problem-solving approach, leverages experience from similar events to aid in resolving new issues. Unlike direct problem analysis, it serves

as an auxiliary and facilitative tool rather than as a means of simplification of the problem directly. Current mainstream reward generation follows a consistent paradigm: generate a proxy, receive interaction feedback, and refine the proxy. As shown in Eq. 1.

$$R = \mathcal{O}(\mathcal{P}, \mathcal{F}(\mathcal{P}, \mathcal{L})) \quad (1)$$

Formally, let $\mathcal{P}$ denote the proxy. In L2R (Yu et al. 2023) this corresponds to the model parameters, whereas in Eureka (Ma et al. 2023) it is the set of functions produced directly by the LLM. Let $\mathcal{L}$ be the reflective chain: in Prompt2Walk (Wang et al. 2023b) it captures the statistical association between observed events and executed behaviors, while in Eureka (Ma et al. 2023) it tracks the modulus changes of return values during code training. Given $\mathcal{P}$ and $\mathcal{L}$, the system derives feedback $\mathcal{F}$ via engineering optimization or other applicable procedures. This feedback is applied to $\mathcal{P}$, yielding an updated proxy $\mathcal{Q}$. The proxy that attains the highest environment reward $\mathcal{R}$ is retained.

Analogical reasoning supplies experiential insights that serve as auxiliary objectives $\mathcal{A}$, prompting a principled revision of the preceding paradigm (Eq. 2). Relevant events provide task-specific guidance, and because skills constitute the necessary conditions for these events, their adoption is highly likely to yield effective solutions. The complete framework is depicted in Fig. 2.

$$R = \mathcal{O}(\mathcal{A}(\mathcal{P}), \mathcal{F}(\mathcal{A}(\mathcal{P}), \mathcal{L})) \quad (2)$$

Section 3.2 draws on analogical reasoning to extract relevant events and distill the corresponding skills into prior knowledge. Section 3.3 converts these skills into auxiliary objectives and combines them with the original task goals to derive reward functions, while mitigating the absence of dedicated evaluation metrics. Section 3.4 introduces an integration scheme that harnesses the potential of the resulting reward functions. Eureka’s capacity for open-ended exploration is unconstrained by predefined models or fixed pathways. We therefore adopt it as our baseline to assess the feasibility of incorporating analogical priors.

3.2 Analogical Thinking for Acquiring Priors

Human analogical thinking enables the meaningful connection of existing problems to other activities or models. These connections can even span across species and bridge the divide between living and nonliving entities. Importantly, these cognitive processes are not purely individualistic. They are communicable and understandable to others. In the long term, this mode of thinking is also a capability that robots should emulate. To capture even the basic essence of human analogical thinking, we start by asking how a human would tackle a new task. Typically, the answer looks something like this: identify a familiar concept that can perform a similar action and then adapt its behavior to the new setting.

$$\mathcal{I} = \zeta(\mathcal{T}, \mathcal{S}) \quad (3)$$

As shown in Eq. 3, we formalize this intuition with a mapping ζ that maps task description $\mathcal{T}$ and agent structure $\mathcal{S}$ to a set $\mathcal{I}$ of auxiliary objectives. Such a direct mapping may encounter certain difficulties and lead to deviations.

To address this, we employ related concepts as intermediate proxies. First, the system breaks down $\mathcal{T}$ into individual aspects while also representing $\mathcal{S}$ as distinct features. Using synonyms and associated terms for retrieval, the system identifies a concept whose structure most closely matches part of $\mathcal{T}$ and possesses the corresponding feature of $\mathcal{S}$. Next, it performs self-analysis to identify which profiles are matchable and then determines how to adjust the behavior or the category to more closely align with the selected features. Detailed prompts for analogical association are provided in Fig. 3. The motion pattern of the chosen concept is translated into a concise, high-level instruction that specifies which body parts should move and how. These instructions form the set $\underline{I}$, which, due to its natural occurrence, is highly likely to be well-suited for accomplishing the original task.

We simulate analogy by matching features at the high-level semantic level. Then these abstract cues are converted into low-level movement rules using motor skills. These low-level rules serve as concise annotations for the instructions and can be readily transformed into executable actions. We can establish rules to filter out auxiliary objectives that are challenging for the agent to achieve, while emphasizing a subset of these objectives that exhibit significant physical changes. This approach further enhances the executability of $\mathcal{I}$. Eq. 3 also captures three critical pieces of information for the task: essential information for task execution, beneficial and actionable rules, and structural components that require coordination. The process is illustrated in Fig. 2A.

3.3 Reward Generation without Metrics

We cannot directly use the auxiliary goals derived from the insights. The main obstacle is that prior knowledge lacks an automatic way to become an evaluation function. Some work uses video analysis to judge agent motion, yet combining this with Eureka’s parallel training remains difficult. Furthermore, the actions implied by the insights and the criteria used to judge them both come from the same source. We instead use current evaluators to measure the gain from prior information.

Fig. 2 B outlines two parallel branches. The first branch keeps the main-task objective active. It first produces a baseline reward r_m (m for main task) and then invokes Eureka to refine it into the optimized reward R_m . The second branch freezes r_m and replaces the task objective with a single piece of prior knowledge. Eureka now searches for an auxiliary reward R_{ak}^n , where k indexes the k -th prior tip and n marks the iteration step, that maximizes the combined reward $r_m + R_{ak}^n$ while leaving r_m unchanged. After each iteration only R_{ak}^n is updated and immediately re-coupled with the same r_m . Repeating the second branch for every prior tip yields a set of distilled auxiliary rewards. Because the baseline r_m is fixed, any performance gain from $r_m + R_{ak}^n$ can be credited solely to R_{ak}^n . This difference provides a built-in metric and automatically removes the parts of the auxiliary reward that could conflict with the main task.

3.4 Gain-Driven Reward Fusion

Our goal is to use the gain from the auxiliary reward $r_m + R_{ak}^n$ over r_m to improve the main-task reward R_m . The dif-

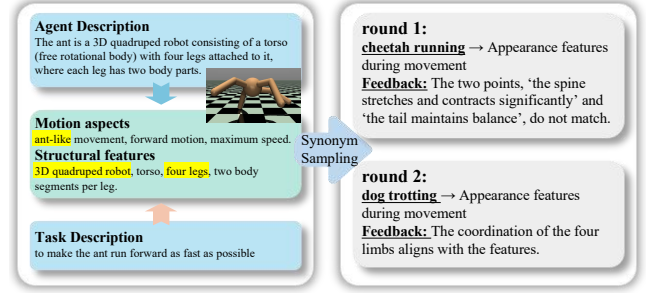


Figure 3: Overview of analogical prior acquisition. The pipeline derives motion skills from real-world objects through three sequential stages: (1) feature decomposition, (2) associative sampling, and (3) exemplar instantiation.

iculty is that each R_{ak}^n is built on the frozen baseline r_m , while R_m has already moved far away from it, so combining them directly produces a behavior gap. We follow one of the remedies that Eureka validated: treat every reward as a weighted sum and let the framework learn the weights. As shown in Fig. 2 C, the final reward is expressed in Eq. 4.

$$R^* = \alpha R_m + \sum_k \beta_k R_{ak}^n \quad (4)$$

$$R^* = \max_i \left\{ R_m + \sum_k \text{softmax}(G_k/T^{(i)}) R_{ak}^n \right\},$$

$$\text{where } T^{(i)} \in \{T_0 - \delta_i, T_0, T_0 + \delta_i\}, \quad \delta_i = \delta_0/2^i. \quad (5)$$

We then search for the scalar weights α and $\{\beta_k\}$ with a weight-tuning routine. Given that most of the reward components above were themselves shaped by this same routine, it is highly probable that merging them through weight tuning will bring further gains to R_m .

We derive auxiliary rewards $r_m + R_{ak}^n$ from prior knowledge and base reward r_m from task objectives. We keep only the auxiliary rewards that outperform r_m and record their gain G_k . Treating G_k as a heuristic, we map it to normalized weights smoothed by a temperature parameter T . In each round i we sample two points, $T_i \pm \delta$. The better performer becomes the new T_{i+1} , and we halve δ to narrow the search. After several iterations we retain the top-N settings for final evaluation. As shown in Eq. 5. We fix $\delta = 0.6$ throughout. Empirically, the heuristic weighting reaches a suitable allocation in only four iterations. Isaac Gym’s acceleration makes the entire reward fusion pipeline highly efficient.

4 Results

4.1 Experiment Settings

Mimicking human analogy is still new and lacks clear benchmarks. A key issue is that analogical association confronts global-local granularity problems. An agent’s limb motion, for example, can be treated as a fine-grained feature.

Yet an isolated limb by itself rarely supplies enough information to function as a meaningful analogical unit. Consequently, although exploiting even finer granularity is appealing, it remains beyond reach at present. We therefore give priority to tasks with clear hierarchical structures, where coarse and fine levels are naturally present, in order to verify the feasibility of our approach. To this end, we adopt several environments in which Eureka already excels. We believe that raising performance further in these settings can strongly support the effectiveness of the method we propose.

In Isaac Gym’s MuJoCo tasks, Humanoid and Ant closely resemble their real-world counterparts and offer limb-level granularity, whereas the FrankaCabinet arm task emphasizes local, low-DOF manipulation. This progression (Humanoid $\rightarrow$ Ant $\rightarrow$ FrankaCabinet) illustrates a clear shift in emphasis: the task bias moves from predominantly global to predominantly local granularity. All three environments are among Eureka’s strongest performers.

In our experiments we modified Eureka, using GPT-4 for code generation and GPT-4o for all non-code components (FrankaCabinet uses GPT-4o for both). We kept the original hyperparameters, e.g., N iterations and K code samples per iteration. For evaluation we compare sparse reward, human reward, L2R (Yu et al. 2023), Eureka (Ma et al. 2023), and ROSKA (Huang et al. 2024). Given the instability of reward generation in Eureka, we ran each experiment five times and kept the best trial for analysis. Backed by Isaac Gym’s acceleration, the pipeline is resource-efficient and fully automated, so the time and compute costs remain modest.

4.2 Performance

Here we outline the baselines. **Sparse**: Binary success indicators supplied by the benchmark, with exact forms that vary by task. **Human**: Expert-shaped dense rewards included with the benchmark. **L2R**: A library of manually defined MPC-based reward primitives whose parameters are set by an LLM interacting with the environment. **Eureka**: Generates full reward functions from scratch, offering greater generality than template-based or task-decomposition approaches. **ROSKA** (Huang et al. 2024): Enhances Eureka with EMA-style (Exponential Moving Average) policy updates. The results in Table 1 follow the metrics reported by ROSKA (Huang et al. 2024) and Eureka (Ma et al. 2023). Our method outperforms Eureka by more than 20 percent.

4.3 Human Preference Alignment

The agent’s action space covers varied motion patterns. By deriving every motion requirement from a single reference target, our method yields a unified movement style, a trend confirmed in the experiments. Eureka-HF incorporates human-feedback reflection and aligns better with human intent but falls short of original Eureka. Our approach, free of human feedback yet grounded in human-like learning, produces policies closer to human preference and cleaner across Humanoid, Ant, and FrankaCabinet tasks, outperforming both Eureka and expert rewards (Table 2).

As Fig. 4 shows, our method yields smoother, more efficient motion across Humanoid, Ant, and FrankaCabinet tasks. On Ant, Eureka and human rewards reduce the agent

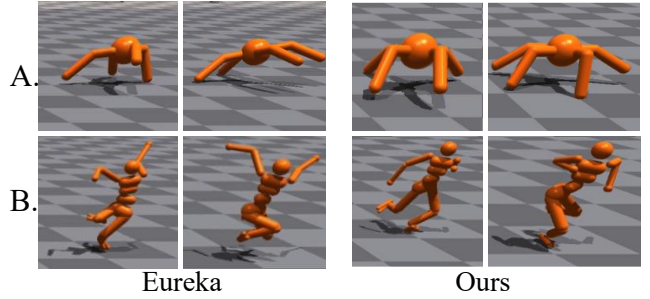


Figure 4: Qualitative analysis shows that our method yields policies closer to human preferences without human feedback; see the appendix for details.

to a tripod gait, cutting DOF and producing sluggish motion. On Humanoid, prior knowledge of the rigid body yields natural, well-coordinated limb motion. On FrankaCabinet, our method constrains the excessive DOF to the effective joints, chiefly the upper arm with supplementary palm adjustments shown in the appendix. These qualitative gains surpass Eureka-HF without extra human feedback and confirm that LLM-guided embodied intelligence can target valuable motion patterns instead of searching them blindly. As shown, our method improves agent performance by aligning the motion of each body part with human-like coordination. Analogical skill transfer thus refines both motion style and task reward.

This raises the question of whether our reward resembles expert-crafted ones. In fact, our reward is a weighted sum of prior-skill components. Unlike the LLM-leakage concern studied in Eureka, the evolutionary path of these components is distinct, owing to the larger set of parallel auxiliary objectives.

Eureka measures the correlation between its reward and the human reward by comparing the two value sequences across all transitions from a fixed policy. In Table 3 we report the correlation of the best reward function, averaged over random seeds. Our method shows markedly lower correlations on tasks where it performs well. Conversely, Eureka’s low correlation on FrankaCabinet stems from poor performance caused by excessive DOF. This divergence arises because prior knowledge steers exploration into distinct regions, altering the transition distribution.

4.4 Heuristic Reward Fusion

The heuristic search for the optimal temperature T narrows the space around high-performing anchors across iterations. We test four weight-generation strategies. 1. “all”: every auxiliary reward is used. 2. “group”: rewards are grouped by body part, weighted internally, then merged with the baseline at equal weight. 3. “best”: only the top reward per body part is retained. 4. $*_r$: inverse weights, set to inverse weights by $-T$, applied to “all, group, and best”.

Fig. 5a–c shows the results on Ant, Humanoid, and FrankaCabinet. All weight variants outperform the baseline R_m (red lines). Despite FrankaCabinet’s limited diversity (one reward per group), the fused policy still surpasses the baseline. Fig. 5d highlights the contribution of auxiliary

	Ant	Humanoid	FrankCabinet
Sparse	6.59 ± 1.44	5.12 ± 0.49	0.0007 ± 0.001
Human	10.35 ± 0.12	6.93 ± 1.38	0.10 ± 0.05
L2R (Yu et al. 2023)	$5.12 \pm -$	$1.42 \pm -$	$0.30 \pm -$
Eureka (Ma et al. 2023)	10.25 ± 1.31	7.24 ± 0.64	0.31 ± 0.22
ROSKA (Huang et al. 2024)	12.07 ± 0.60	9.10 ± 1.06	0.85 ± 0.19
Ours	12.77 ± 0.93	9.60 ± 0.79	0.91 ± 0.06

Table 1: Comparison across three tasks (mean $\pm$ std)

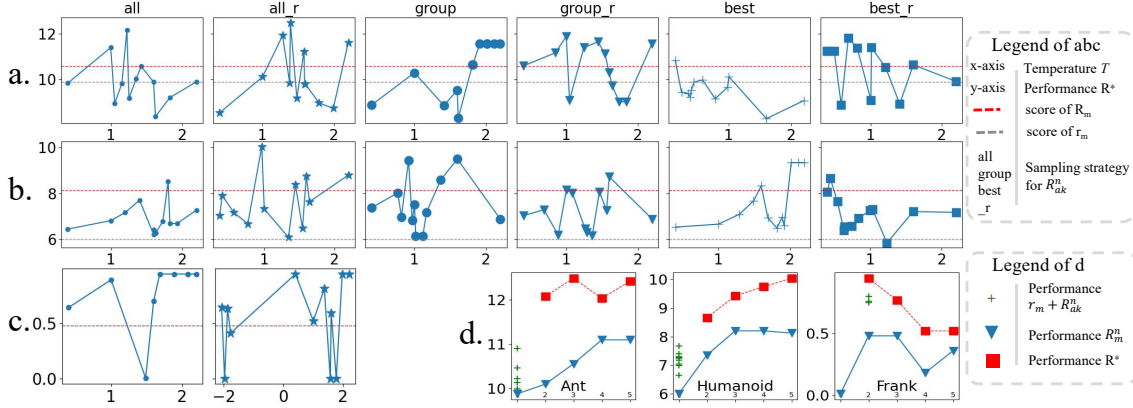


Figure 5: **Reward-Fusion Performance R^* versus Temperature T** : (a)–(c) depict the performance of reward fusion with varying weight settings across normalized temperature T in the Ant, Humanoid, and FrankaCabinet environments. **Performance R^* versus additional iterations i** : (d) shows the changes in auxiliary rewards (green pluses), baseline rewards (blue triangles), and fused results (red squares). For more details, see Section 4.4.

Env	F	Eureka	Eureka-HF	Ours
Ant	V	11.78	-	12.49
Humanoid	V	7.97	5.58	10.03
FrankCabinet	$open$	0.48	-	0.97

Table 2: Aligns with human preference; F : fitness function; V : forward velocity; $Open$: cabinet door opened

Correlation $\downarrow$	Eureka	Ours
Ant	0.97	0.90
Humanoid	0.97	0.84
FrankCabinet	0.47	0.56

Table 3: the correlation between policy trajectories trained with its reward functions and those designed by experts. Lower correlation $\rightarrow$ reduced LLM-knowledge leakage

wards. Blue triangles track baseline reward R_m^n over iterations; red squares mark final performance after fusion R^* ; green pluses show auxiliary-reward performance $r_m + R_{ak}^n$. Auxiliary rewards are generated whenever the R_m^n exceeds the average threshold. The corresponding baseline value R_m^n is recorded as “born_score” (gray lines in a–c). Our heuristic fusion consistently improves results across all weight settings.

5 Ablation Studies

5.1 Naive Experience

We first ran Eureka on the Humanoid task, inserting prior knowledge into the prompt. The results were poor: fewer

than half of the generated rewards responded to the auxiliary information, and most of those that did misinterpreted it or engaged superficially. When a response was accurate, the reward intertwined main-task and auxiliary components in ways that subsequent iterations could not disentangle, leaving performance unchanged. Overall, direct incorporation of prior knowledge failed to yield useful rewards.

5.2 One-Piece Control

To mitigate confusion, we adopt the control variables method: each Eureka iteration incorporates only one prior-derived auxiliary reward. Although main and auxiliary components remain entangled, performance consistently improves. Figure 6(A) shows the gain when an auxiliary reward is active; Figure 6(B) shows the gain persists into the next iteration. These results confirm that analogically derived rewards benefit the main task. Yet most auxiliary rewards vanish after the second iteration, indicating that the method boosts performance transiently but fails to retain and refine them. Sustaining and evolving these rewards cannot be achieved by the control-variable method alone.

5.3 Distillation for Reward Fusion

The auxiliary rewards are built upon the baseline, yet the main reward has already diverged far from it. This divergence produces behaviour mismatches between the two sources, which in turn undermines the effectiveness of distillation. We first attempted a multi-teacher knowledge distillation approach within the framework of reinforcement

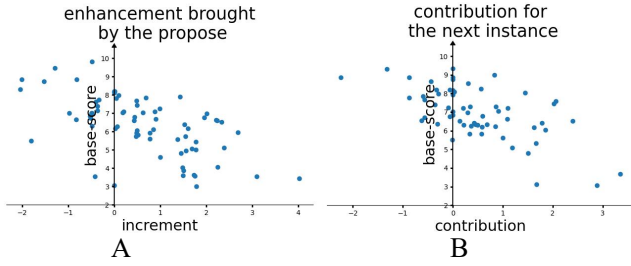


Figure 6: **Ablation study of module B.** A: Auxiliary-reward $r_m + R_{ak}^n$ gains relative to the base reward r_m . The y-axis gives base-reward performance; the x-axis shows the gain (negative values indicate losses). B: Gains persist into the next iteration; the y-axis shows the current reward $r_m + R_{ak}^n$ performance.

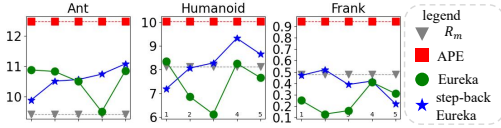


Figure 7: **Red squares** show our method’s performance. **Gray triangles** represent the selected baseline. **Blue stars** indicate “step-back Eureka” over multiple iterations. **Green dots** show Eureka with extended iterations.

learning. In this setup, the teacher models were defined by the reward functions $r_m + R_{ak}^n$, and the student model utilized R_m . Apart from the differences in reward functions, all other configurations of the models were identical. We explored two distinct methods: one involved calculating similarity through feature extraction, and the other focused on collecting high-quality trajectories generated by the teacher models. However, the outcomes of these methods were not satisfactory.

5.4 Weighted Distillation

We suspect that the auxiliary rewards, each tailored to a distinct body part, conflict because they target different objectives. Treating all parts uniformly obscures these differences. To test this, we re-ran the distillation after assigning explicit weights to the body segments. For each auxiliary reward, we manually identified the relevant joints and distilled only the corresponding slice of the action distribution produced by the policy. The procedure, however, still failed to converge and yielded no viable policy. Further exploration of these challenges extends beyond the current paper’s scope. We instead explored a more straightforward baseline approach. Interestingly, this simpler method demonstrated promising results. It showed better outcomes compared to the more complex approaches.

5.5 Step-back and Iteration

The Step-Back method (Zheng et al. 2024) injects domain knowledge into context and improves hard reasoning tasks. Step-back retrieves relevant knowledge domains, whereas our association builds cue-based links among individual el-

ements. To test whether holistic contexts derived from association can help, we run “step-back Eureka” that embeds associative rules in the prompt. Starting from the baseline of Section 4.1, both step-back Eureka and extended Eureka iterations yield minor, unstable gains (Fig. 7). Step-back Eureka edges ahead slightly, yet the improvement is within run-to-run noise. We therefore classify this approach with the “Naive Experience” 5.1 experiments.

Introducing prior knowledge raises LLM interactions, and Eureka claims more iterations boost reward quality. To test whether gains are merely iteration-driven, we seeded the best baseline and increased iterations alone. The blue dotted line in Fig. 7 shows performance quickly plateaus, confirming that extra iterations yield diminishing returns.

5.6 Discussion

Our approach emulates human analogy: it extracts motion tips through association and augments the reward objective, a direction that is still seldom explored. Our experience has identified three obstacles. First, an ideal analogical model needs to dynamically shift its focus between global and local body features as task difficulty increases. Such adaptability is indispensable in complex environments. Second, a portion of valuable motion tips is wasted during reward generation or fusion because existing evaluation functions deviate from the intended contribution of auxiliary objectives. Moreover, tasks with a clear sequential structure demand yet-to-be-found embedding rules for subtasks, and no single rule transfers across them. We argue that the decisive step is to devise evaluation functions tailored to each tip, functions that remain compatible with parallel training and avoid over-reliance on vision-centric large models. In the absence of such functions, our framework now offers a practical way for validating the value of analogical augmentation. It offers a benchmark for future research.

6 Conclusion

Endowing agents with new skills is a hot topic in the field of artificial intelligence. Designing reward functions using large language models (LLMs) among an effective solution to achieve this goal. However, existing methods have certain deficiencies in terms of transparency and result controllability. Integrating human experiential knowledge into the skill learning process of robots is a natural and intuitive choice. This approach not only improves the learning efficiency of agents but also enhances the interpretability of their behaviors. Using associative methods to identify relevant models significantly aids problem-solving and clearly distinguishes from common LLM fine-tuning approaches like Chain of Thought (CoT) and context-based methods, making it worthy of broader exploration. This approach offers a practical route to augment LLM-based reward objectives and to tap the model’s latent knowledge. Yet, whether applied directly or through step-back prompting, such mining still falls short of delivering substantial, reliable gains in reward design.

References

- Ahn, M.; Brohan, A.; Brown, N.; Chebotar, Y.; Cortes, O.; David, B.; Finn, C.; Gopalakrishnan, K.; Hausman, K.; Herzog, A.; Ho, D.; Hsu, J.; Ibarz, J.; Ichter, B.; Irpan, A.; Jang, E.; Ruano, R. M. J.; Jeffrey, K.; Jesmonth, S.; Joshi, N. J.; Julian, R. C.; Kalashnikov, D.; Kuang, Y.; Lee, K.-H.; Levine, S.; Lu, Y.; Luu, L.; Parada, C.; Pastor, P.; Quiambao, J.; Rao, K.; Rettinghouse, J.; Reyes, D. M.; Sermanet, P.; Sievers, N.; Tan, C.; Toshev, A.; Vanhoucke, V.; Xia, F.; Xiao, T.; Xu, P.; Xu, S.; and Yan, M. 2022. Do As I Can, Not As I Say: Grounding Language in Robotic Affordances. In *Conference on Robot Learning*.
- Bhambri, S.; Bhattacharjee, A.; Kalwar, D.; Guan, L.; Liu, H.; and Kambhampati, S. 2024. Extracting Heuristics from Large Language Models for Reward Shaping in Reinforcement Learning. ArXiv:2405.15194.
- Biza, O.; Wang, D.; Jr., R. P.; van de Meent, J.; and Wong, L. L. S. 2021. Action Priors for Large Action Spaces in Robotics. ArXiv:2101.04178.
- Cao, Y.; Zhao, H.; Cheng, Y.; Shu, T.; Chen, Y.; Liu, G.; Liang, G.; Zhao, J.; Yan, J.; and Li, Y. 2024. Survey on Large Language Model-Enhanced Reinforcement Learning: Concept, Taxonomy, and Methods. *IEEE Transactions on Neural Networks and Learning Systems*, 1–21.
- Carta, T.; Oudeyer, P.-Y.; Sigaud, O.; and Lamprier, S. 2022. EAGER: Asking and Answering Questions for Automatic Reward Shaping in Language-guided RL. *Advances in Neural Information Processing Systems*, 35: 12478–12490.
- Chu, K.; Zhao, X.; Weber, C.; Li, M.; and Wermter, S. 2023. Accelerating Reinforcement Learning of Robotic Manipulations via Feedback from Large Language Models. ArXiv:2311.02379.
- Cui, J.; Liu, T.; Meng, Z.; Yu, J.; Song, R.; Zhang, W.; Zhu, Y.; and Huang, S. 2025. GROVE: A Generalized Reward for Learning Open-Vocabulary Physical Skill. ArXiv:2504.04191.
- Davidson, G.; Todd, G.; Togelius, J.; Gureckis, T. M.; and Lake, B. M. 2025. Goals as reward-producing programs. *Nature Machine Intelligence*, 7(2): 205–220.
- Ding, T. 2024. MobileAgent: enhancing mobile control via human-machine interaction and SOP integration. ArXiv:2401.04124.
- Ding, Y.; Zhang, X.; Amiri, S.; Cao, N.; Yang, H.; Kaminiski, A.; Esselink, C.; and Zhang, S. 2023. Integrating action knowledge and LLMs for task planning and situation handling in open worlds. *Auton. Robots*, 47(8): 981–997.
- Du, Y.; Watkins, O.; Wang, Z.; Colas, C.; Darrell, T.; Abbeel, P.; Gupta, A.; and Andreas, J. 2023. Guiding pre-training in reinforcement learning with large language models. In *Proceedings of the 40th International Conference on Machine Learning*.
- Fan, L.; Krishnan, D.; Isola, P.; Katabi, D.; and Tian, Y. 2023. Improving CLIP Training with Language Rewrites. ArXiv:2305.20088.
- Hazra, R.; Sygkounas, A.; Persson, A.; Loutfi, A.; and Martires, P. Z. D. 2025. REvolve: Reward Evolution with Large Language Models using Human Feedback. ArXiv:2406.01309.
- Huang, C.; Chang, Y.; Lin, J.; Liang, J.; Zeng, R.; and Li, J. 2024. Efficient Language-instructed Skill Acquisition via Reward-Policy Co-Evolution. ArXiv:2412.13492.
- Huang, W.; Wang, C.; Zhang, R.; Li, Y.; Wu, J.; and Fei-Fei, L. 2023. VoxPoser: Composable 3D Value Maps for Robotic Manipulation with Language Models. ArXiv:2307.05973.
- Kwon, M.; Xie, S. M.; Bullard, K.; and Sadigh, D. 2023. Reward Design with Language Models. ArXiv:2303.00001.
- Li, H.; Yang, X.; Wang, Z.; Zhu, X.; Zhou, J.; Qiao, Y.; Wang, X.; Li, H.; Lu, L.; and Dai, J. 2024. Auto MC-Reward: Automated Dense Reward Design with Large Language Models for Minecraft. ArXiv:2312.09238.
- Liang, J.; Xia, F.; Yu, W.; Zeng, A.; Arenas, M. G.; Atarian, M.; Bauzá, M.; Bennice, M.; Bewley, A.; Dostmohamed, A.; Fu, C.; Gileadi, N.; Giustina, M.; Gopalakrishnan, K.; Hasenclever, L.; Humplik, J.; Hsu, J.; Joshi, N. J.; Jyenis, B.; Kew, C.; Kirmani, S.; Lee, T.-W. E.; Lee, K.-H.; Michaely, A. H.; Moore, J.; Oslund, K.; Rao, D.; Ren, A.; Tabanpour, B.; Vuong, Q. H.; Wahid, A.; Xiao, T.; Xu, Y.; Zhuang, V.; Xu, P.; Frey, E.; Caluwaerts, K.; Zhang, T.-Y.; Ichter, B.; Tompson, J.; Takayama, L.; Vanhoucke, V.; Shafraan, I.; Mataric, M.; Sadigh, D.; Heess, N. M. O.; Rao, K.; Stewart, N.; Tan, J.; and Parada, C. 2024a. Learning to Learn Faster from Human Feedback with Language Model Predictive Control. ArXiv:2402.11450.
- Liang, W.; Wang, S.; Wang, H.-J.; Bastani, O.; Jayaraman, D.; and Ma, Y. J. 2024b. EurekaVerse: Environment Curriculum Generation via Large Language Models. ArXiv:2411.01775.
- Lin, B. Y.; Fu, Y.; Yang, K.; Ammanabrolu, P.; Brahman, F.; Huang, S.; Bhagavatula, C.; Choi, Y.; and Ren, X. 2023. SwiftSage: A Generative Agent with Fast and Slow Thinking for Complex Interactive Tasks. ArXiv:2305.17390.
- Ma, Y. J.; Liang, W.; Wang, G.; Huang, D.-A.; Bastani, O.; Jayaraman, D.; Zhu, Y.; Fan, L.; and Anandkumar, A. 2023. Eureka: Human-Level Reward Design via Coding Large Language Models. ArXiv:2310.12931.
- Mandi, Z.; Jain, S.; and Song, S. 2024. RoCo: Dialectic Multi-Robot Collaboration with Large Language Models. *2024 IEEE International Conference on Robotics and Automation (ICRA)*, 286–299.
- Narin, A. 2024. Evolutionary Reward Design and Optimization with Multimodal Large Language Models. *Association for Computational Linguistics*, 202–208.
- Rana, K.; Haviland, J.; Garg, S.; Abou-Chakra, J.; Reid, I. D.; and Sünderhauf, N. 2023. SayPlan: Grounding Large Language Models using 3D Scene Graphs for Scalable Task Planning. In *Conference on Robot Learning*.
- Ren, A. Z.; Dixit, A.; Bodrova, A.; Singh, S.; Tu, S.; Brown, N.; Xu, P.; Takayama, L.; Xia, F.; Varley, J.; Xu, Z.; Sadigh, D.; Zeng, A.; and Majumdar, A. 2023. Robots That Ask For Help: Uncertainty Alignment for Large Language Model Planners. In *Conference on Robot Learning*.

Ryu, K.; Liao, Q.; Li, Z.; Sreenath, K.; and Mehr, N. 2024. CurricuLLM: Automatic Task Curricula Design for Learning Complex Robot Skills using Large Language Models. ArXiv:2409.18382.

Sha, H.; Mu, Y.; Jiang, Y.; Chen, L.; Xu, C.; Luo, P.; Li, S. E.; Tomizuka, M.; Zhan, W.; and Ding, M. 2023. LanguageMPC: Large Language Models as Decision Makers for Autonomous Driving. ArXiv:2310.03026.

Shukla, Y.; Gao, W.; Sarathy, V.; Velasquez, A.; Wright, R.; and Sinapov, J. 2024. LgTS: Dynamic Task Sampling using LLM-generated Sub-Goals for Reinforcement Learning Agents. *Proceedings of the 23rd International Conference on Autonomous Agents and Multiagent Systems*, 1736–1744.

Singh, U.; Bhattacharyya, P.; and Namboodiri, V. P. 2024. LGR2: Language Guided Reward Relabeling for Accelerating Hierarchical Reinforcement Learning. ArXiv:2406.05881.

Singh, U.; and Namboodiri, V. P. 2023. CRISP: Curriculum inducing Primitive Informed Subgoal Prediction for Hierarchical Reinforcement Learning. ArXiv:2304.03535.

Skreta, M.; Zhou, Z.; Yuan, J. L.; Darvish, K.; Aspuru-Guzik, A.; and Garg, A. 2024. RePlan: Robotic Replanning with Perception and Language Models. ArXiv:2401.04157.

Sun, S.; Liu, R.; Lyu, J.; Yang, J.-W.; Zhang, L.; and Li, X. 2024. A Large Language Model-Driven Reward Design Framework via Dynamic Feedback for Reinforcement Learning. ArXiv:2410.14660.

Tang, S.; King, I.; and Xu, Z. 2025. Combining Explicit Priors and Set Attention Driven Implicit Priors for Demonstration-Guided Reinforcement Learning. In *Neural Information Processing*.

Tang, Y.; Yu, W.; Tan, J.; Zen, H.; Faust, A.; and Harada, T. 2023. SayTap: Language to Quadrupedal Locomotion. ArXiv:2306.07580.

Tirumala, D.; Galashov, A.; Noh, H.; Hasenclever, L.; Pascanu, R.; Schwarz, J.; Desjardins, G.; Czarnecki, W. M.; Ahuja, A.; Teh, Y. W.; and Heess, N. M. O. 2020. Behavior Priors for Efficient Reinforcement Learning. *J. Mach. Learn. Res.*, 23: 221:1–221:68.

Vemprala, S.; Bonatti, R.; Bucker, A.; and Kapoor, A. 2023. ChatGPT for Robotics: Design Principles and Model Abilities. ArXiv:2306.17582.

Verma, M.; and Metcalf, K. 2022. Symbol Guided Hindsight Priors for Reward Learning from Human Preferences. ArXiv:2210.09151.

Verma, M.; and Susa, R. M. 2024. Hindsight PRIORS for Reward Learning from Human Preferences. In *ICLR*.

Wang, G.; Xie, Y.; Jiang, Y.; Mandlekar, A.; Xiao, C.; Zhu, Y.; Fan, L.; and Anandkumar, A. 2023a. Voyager: An Open-Ended Embodied Agent with Large Language Models. ArXiv:2305.16291.

Wang, Y.-J.; Zhang, B.; Chen, J.; and Sreenath, K. 2023b. Prompt a Robot to Walk with Large Language Models. ArXiv:2309.09969.

Wu, J.; Antonova, R.; Kan, A.; Lepert, M.; Zeng, A.; Song, S.; Bohg, J.; Rusinkiewicz, S.; and Funkhouser, T. 2023.

TidyBot: Personalized Robot Assistance with Large Language Models. *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 3546–3553.

Wu, X. 2024. From Reward Shaping to Q-Shaping: Achieving Unbiased Learning with LLM-Guided Knowledge. ArXiv:2410.01458.

Xie, T.; Zhao, S.; Wu, C. H.; Liu, Y.; Luo, Q.; Zhong, V.; Yang, Y.; and Yu, T. 2023. Text2Reward: Automated Dense Reward Function Generation for Reinforcement Learning. ArXiv:2309.11489.

Xu, M.; Huang, P.; Yu, W.; Liu, S.; Zhang, X.; Niu, Y.; Zhang, T.; Xia, F.; Tan, J.; and Zhao, D. 2023. Creative Robot Tool Use with Large Language Models. ArXiv:2310.13065.

Xu, M.; Veloso, M.; and Song, S. 2022. AS-PiRe: Adaptive Skill Priors for Reinforcement Learning. ArXiv:2209.15205.

Yu, W.; Gileadi, N.; Fu, C.; Kirmani, S.; Lee, K.-H.; Arenas, M. G.; Chiang, H.-T. L.; Erez, T.; Hasenclever, L.; Humpalik, J.; Ichter, B.; Xiao, T.; Xu, P.; Zeng, A.; Zhang, T.; Heess, N. M. O.; Sadigh, D.; Tan, J.; Tassa, Y.; and Xia, F. 2023. Language to Rewards for Robotic Skill Synthesis. ArXiv:2306.08647.

Zeng, Y.; Mu, Y.; and Shao, L. 2024. Learning Reward for Robot Skills Using Large Language Models via Self-Alignment. ArXiv:2405.07162.

Zheng, H. S.; Mishra, S.; Chen, X.; Cheng, H.-T.; Chi, E. H.; Le, Q. V.; and Zhou, D. 2024. Take a Step Back: Evoking Reasoning via Abstraction in Large Language Models. ArXiv:2310.06117.

7 Reproducibility Checklist

1. This paper:
 - Includes a conceptual outline and/or pseudocode description of AI methods introduced (yes/partial/no/NA)
 - Clearly delineates statements that are opinions, hypotheses, and speculation from objective facts and results (yes/no)
 - Provides well-marked pedagogical references for less-familiar readers to gain background necessary to replicate the paper (yes/no)
2. Does this paper make theoretical contributions? (yes/no)
If yes, please complete the list below:
 - All assumptions and restrictions are stated clearly and formally. (yes/partial/no)
 - All novel claims are stated formally (e.g., in theorem statements). (yes/partial/no)
 - Proofs of all novel claims are included. (yes/partial/no)
 - Proof sketches or intuitions are given for complex and/or novel results. (yes/partial/no)
 - Appropriate citations to theoretical tools used are given. (yes/partial/no)
 - All theoretical claims are demonstrated empirically to hold. (yes/partial/no/NA)

- All experimental code used to eliminate or disprove claims is included. (yes/no/NA)
3. Does this paper rely on one or more datasets? (yes/no)
If yes, please complete the list below:
- A motivation is given for why the experiments are conducted on the selected datasets. (yes/partial/no/NA)
 - All novel datasets introduced in this paper are included in a data appendix. (yes/partial/no/NA)
 - All novel datasets introduced in this paper will be made publicly available upon publication with a license allowing free research use. (yes/partial/no/NA)
 - All datasets drawn from the existing literature are accompanied by appropriate citations. (yes/no/NA)
 - All datasets drawn from the existing literature are publicly available. (yes/partial/no/NA)
 - Datasets that are not publicly available are described in detail, with justification. (yes/partial/no/NA)
4. Does this paper include computational experiments? (yes/no)
If yes, please complete the list below:
- Number/range of values tried per (hyper-)parameter and selection criteria are reported. (yes/partial/no/NA)
 - Code for data preprocessing is included in the appendix. (yes/partial/no)
 - Source code for conducting and analyzing experiments is included. (yes/partial/no)
 - Code will be released publicly upon publication with a permissive license. (yes/partial/no)
 - Code includes comments with implementation details and paper references. (yes/partial/no)
 - Seed setting methods for stochastic algorithms are described. (yes/partial/no/NA)
 - Computing infrastructure (hardware/software specs) is reported. (yes/partial/no)
 - Evaluation metrics are formally described with motivations. (yes/partial/no)
 - Number of runs per result is specified. (yes/no)
 - Performance analysis includes variation, confidence, or distributions. (yes/no)
 - Significance of performance differences is assessed with statistical tests. (yes/partial/no)
 - Final (hyper-)parameter settings are listed. (yes/partial/no/NA)